05주차 과제 제출 답안

AI 데이터개발과 MLOps | 전영준 / 202373020

1. 수용·폐기 이벤트의 도착 시점 watermark 계산

본인 산출물의 watermark_delay="10 minutes"이고 window_size="10 minutes"이며, watermark는 현재 벽시계 시각이 아니라 도착 전에 관측된 최대 이벤트 시각에서 지연 허용값을 빼서 계산한다. LATE_OK의 event_time="22:04"에 대해 이전 파일의 최대 이벤트 시각 22:09에서 10분을 빼면 21:59로, watermark_at_arrival="21:59 (= 22:09 - 10m)"와 일치하고 창 [22:00, 22:10)의 끝이 이 기준보다 뒤라 decision="수용"이며 강남구 건수가 3에서 4로 갱신됐다. LATE_DROP의 event_time="21:49"에 대해 이전 파일까지의 최대 이벤트 시각 22:18에서 10분을 빼면 22:08로, watermark_at_arrival="22:08 (= 22:18 - 10m)"와 일치하고 창 [21:40, 21:50)의 끝이 이 기준보다 앞이라 decision="폐기"이다.

2. 집계에 나타나지 않는 폐기를 운영에서 감지하는 방법

LATE_DROP은 event_time="21:49", watermark_at_arrival="22:08 (= 22:18 - 10m)", decision="폐기"이며, [21:40, 21:50) 창이 집계에 없다는 사실만으로 운영에서 폐기 발생을 확실히 감지할 수는 없다. Spark의 진행 메트릭 stateOperators[].numRowsDroppedByWatermark를 수집해야 하며, 본인 파일의 dropped_by_watermark_cumulative는 batch1.json=0, batch2.json=0, batch3.json=1로 증가했다. 운영에서는 query ID·run ID·batch ID·상태 연산자별로 배치 지표를 한 번씩 저장해 폐기 건수와 추세에 경보를 걸고, 입력량·이벤트 지연 분포도 함께 확인해야 한다.

3. “우리 대시보드 수치는 실시간이라 정확하다”에 대한 반박

본인 파일의 window_size="10 minutes"는 발생 시각 기준 10분 창을 뜻하고, watermark_delay="10 minutes"는 지연 허용 정책이므로 빠르게 갱신된다는 사실만으로 수치의 완전성이나 최종 확정성을 보장하지 않는다. event_time="22:04"인 LATE_OK는 watermark_at_arrival="21:59 (= 22:09 - 10m)"에서 decision="수용"으로 집계를 3에서 4로 바꾼 반면, event_time="21:49"인 LATE_DROP은 watermark_at_arrival="22:08 (= 22:18 - 10m)"에서 decision="폐기"되어 실제 입력 1건이 집계에 빠졌다. 따라서 대시보드는 창 크기·지연 허용값·폐기 건수와 잠정치 여부를 함께 표시해야 하며, 실시간이라는 이유만으로 모든 발생 이벤트가 정확히 반영됐다고 주장할 수 없다.

실행 근거: Python 3.13 / PySpark 4.1.2 / Java 17, 교수자 원본 코드 실행. 고정 시각의 가상 민원 14건을 실제 Spark 파일 소스로 처리했으며 실시간 민원 API 호출은 아님. JSON 생성 시각: 2026-10-04T09:59:20.373715+00:00.

필수 첨부: practice/chapter5/data/output/ch5_late_event_report.json (생성 원본 그대로). 본문은 문항별 3문장.